

Smart Contract Security Audit Report

TSwap Protocol

Abinash Paudel

July 8, 2026



Contents

1	Executive Summary	4
1.1	Scope	4
1.2	Methodology	4
1.3	Protocol Details	4
1.4	Severity Classification	4
1.5	Summary of Findings	5
2	High Findings	6
2.1	[H-1] <code>TSwapPool::deposit()</code> is missing deadline check causing the transaction to complete even if the deadline has passed	6
2.1.1	Description	6
2.1.2	Location	6
2.1.3	Impact	6
2.1.4	Proof of Concept	6
2.1.5	Recommendation	6
2.2	[H-2] Incorrect fee calculation in <code>TSwapPool::getInputAmountBasedOnOutput()</code> causes the protocol to take too many tokens from users	7
2.2.1	Description	7
2.2.2	Location	7
2.2.3	Impact	7
2.2.4	Proof of Concept	7
2.2.5	Recommendation	7
2.3	[H-3] Invariant $x * y = k$ is broken in <code>TSwapPool::_swap()</code> due to an extra reward transfer every <code>SWAP_COUNT_MAX</code> swaps	8
2.3.1	Description	8
2.3.2	Location	8
2.3.3	Impact	8
2.3.4	Proof of Concept	8
2.3.5	Recommendation	8
3	Low Findings	9
3.1	[L-1] <code>TSwapPool::LiquidityAdded</code> event has parameters out of order	9
3.1.1	Description	9
3.1.2	Impact	9
3.1.3	Recommendation	9
3.2	[L-2] <code>TSwapPool::swapExactInput()</code> always returns 0	10
3.2.1	Description	10
3.2.2	Impact	10
3.2.3	Proof of Concept	10
3.2.4	Recommendation	10
4	Informational Findings	11
4.1	[I-1] <code>PoolFactory::PoolFactory__PoolDoesNotExist</code> is unused and should be removed	11
4.2	[I-2] Lacking zero-address check in <code>PoolFactory</code> constructor	11
4.3	[I-3] <code>PoolFactory::createPool</code> should use <code>.symbol()</code> instead of <code>.name()</code>	11
4.4	[I-4] Magic numbers should be replaced with named constants	11
4.5	[I-5] <code>TSwapPool::swapExactInput()</code> should be <code>external</code>	11
4.6	[I-6] <code>TSwapPool::swapExactOutput()</code> is missing <code>NatSpec</code> for the <code>deadline</code> parameter	11

4.7	[I-7] TSwapPool::_swap() uses deprecated contract-type equality comparison . .	12
5	Gas / Static Analysis Notes (Aderyn)	13
6	Disclaimer	14

1 Executive Summary

Project: TSwap **Repository:** [\[link to repo\]](#) **Commit hash audited:** `f426f57731208727addc20adb72cb7f5bf29dc03`
Audit dates: [start date] – “today” **Auditor(s):** Abinash Paudel

TSwap is an automated market maker (AMM) protocol, structurally similar to Uniswap V1, allowing users to swap between WETH and any ERC20 pool token, and to add/remove liquidity via a factory-deployed pool per token.

1.1 Scope

File	SLOC	Description
<code>src/PoolFactory.sol</code>	35	Deploys and tracks one liquidity pool per ERC20 token
<code>src/TSwapPool.sol</code>	315	Core AMM logic: swaps, deposits, withdrawals, pricing

Total nSLOC: 350 **Complexity score:** 174 **External interactions:** Many ERC20 tokens (arbitrary, including fee-on-transfer/rebasing/deflationary tokens not explicitly excluded) **Test coverage at time of audit:** 40.91%

1.2 Methodology

- Manual line-by-line review
- Static analysis with [Aderyn](#)
- Fuzz / invariant testing with Foundry

1.3 Protocol Details

Fork of existing protocol	Yes — UniswapV1
Multi-chain	No (Ethereum mainnet only)
External oracles	No
External AMMs	No
Off-chain processes (keeper bots, etc.)	No

1.4 Severity Classification

Severity	Definition
Critical	Directly exploitable, leads to loss of funds or full compromise
High	Exploitable under realistic conditions, significant impact
Medium	Exploitable under specific conditions, moderate impact
Low	Edge case or best-practice deviation, minor impact
Informational / Gas	Code quality, gas optimization, no security impact

1.5 Summary of Findings

ID	Title	Severity	Status
H-1	<code>deposit()</code> missing deadline check	High	Open
H-2	Incorrect fee scaling in <code>getInputAmountBasedOnOutput()</code>	High	Open
H-3	$x*y=k$ invariant broken by extra reward transfer in <code>_swap()</code>	High	Open
L-1	<code>LiquidityAdded</code> event parameters out of order	Low	Open
L-2	<code>swapExactInput()</code> always returns 0	Low	Open
I-1	Unused error <code>PoolFactory__PoolDoesNotExist</code>	Informational	Open
I-2	Missing zero-address check in constructor	Informational	Open
I-3	<code>createPool</code> should use <code>.symbol()</code> not <code>.name()</code>	Informational	Open
I-4	Magic numbers used instead of named constants	Informational	Open
I-5	<code>swapExactInput()</code> should be <code>external</code>	Informational	Open
I-6	Missing <code>NatSpec</code> for <code>deadline</code> param	Informational	Open
I-7	Deprecated contract-type equality comparison	Informational	Open

2 High Findings

2.1 [H-1] TSwapPool::deposit() is missing deadline check causing the transaction to complete even if the deadline has passed

Severity: High **Status:** Open

2.1.1 Description

The `deposit` function accepts a `deadline` parameter which, per the documentation, is “the deadline for the transaction to be completed by.” However, this parameter is never used. As a consequence, operations that add liquidity to the pool might execute at unexpected times, under market conditions where the deposit rate is unfavorable.

2.1.2 Location

`src/TSwapPool.sol`

2.1.3 Impact

Transactions could be mined when market conditions are unfavorable for the depositor, even though a deadline parameter was explicitly supplied to guard against this.

2.1.4 Proof of Concept

The `deadline` parameter is accepted but unused anywhere in the function body.

2.1.5 Recommendation

```
1 function deposit(  
2     uint256 wethToDeposit,  
3     uint256 minimumLiquidityTokensToMint,  
4     uint256 maximumPoolTokensToDeposit,  
5     uint64 deadline  
6 )  
7     external  
8 +   revertIfDeadlinePassed(deadline)  
9     revertIfZero(wethToDeposit)  
10    returns (uint256 liquidityTokensToMint)  
11 {}
```

2.2 [H-2] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput()` causes the protocol to take too many tokens from users

Severity: High Status: Open

2.2.1 Description

`getInputAmountBasedOnOutput()` calculates the amount of input tokens required to receive a specific amount of output tokens. The fee calculation scales the amount by 10_000 instead of the correct 1_000.

2.2.2 Location

src/TSwapPool.sol

2.2.3 Impact

The protocol takes roughly 10x more in fees than intended from users on every swap using this path.

2.2.4 Proof of Concept

```
1 uint256 expected =
2     (output * inputReserve * 1000) /
3     ((outputReserve - output) * 997);
4
5 uint256 actual =
6     pool.getInputAmountBasedOnOutput(output, inputReserve, outputReserve);
7
8 assertEq(actual, expected * 10);
```

2.2.5 Recommendation

```
1 - ((inputReserves * outputAmount) * 10000) /
2 + ((inputReserves * outputAmount) * 1000) /
3   ((outputReserves - outputAmount) * 997);
```

2.3 [H-3] Invariant $x * y = k$ is broken in TSwapPool::_swap() due to an extra reward transfer every SWAP_COUNT_MAX swaps

Severity: High Status: Open

2.3.1 Description

Where x is the pool-token balance, y is the WETH balance, and k is their constant product, _swap() sends an extra 1_000_000_000_000_000_000 (1e18) tokens to the swapper once the running swap count reaches SWAP_COUNT_MAX. This disrupts the AMM's core invariant — the ratio between the two balances no longer holds after the bonus payout.

2.3.2 Location

src/TSwapPool.sol

2.3.3 Impact

The protocol's pricing math and accounting become invalid after the bonus fires. A user can repeatedly trigger the bonus payout to drain the contract's WETH balance.

2.3.4 Proof of Concept

```

1 function testBroken() public {
2     vm.startPrank(liquidityProvider);
3     weth.approve(address(pool), 100e18);
4     poolToken.approve(address(pool), 100e18);
5     pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6     vm.stopPrank();
7     uint256 wethAmount = 1e17;
8
9     vm.startPrank(user);
10    poolToken.approve(address(pool), type(uint256).max);
11    poolToken.mint(user, 100e18);
12    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
13    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
14    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
15    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
16    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
17    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
18    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
19    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
20    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
21    int256 startingY = int256(weth.balanceOf(address(pool)));
22    int256 expDeltaY = -int256(wethAmount);
23    pool.swapExactOutput(poolToken, weth, wethAmount, uint64(block.timestamp));
24    vm.stopPrank();
25
26    uint256 endingY = weth.balanceOf(address(pool));
27    int256 actualDeltaY = int256(endingY) - int256(startingY);
28    assertEq(actualDeltaY, expDeltaY);
29 }
```

2.3.5 Recommendation

Remove the extra incentive. If it must be kept, account for it against the invariant explicitly, the same way protocol fees are set aside.

```

1 - swap_count++;
2 - if (swap_count >= SWAP_COUNT_MAX) {
3 -     swap_count = 0;
4 -     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5 - }
```

3 Low Findings

3.1 [L-1] TSwapPool::LiquidityAdded event has parameters out of order

Severity: Low Status: Open

3.1.1 Description

When `LiquidityAdded` is emitted in `_addLiquidityMintAndTransfer`, the logged values are in the wrong order relative to the event's declared parameters.

3.1.2 Impact

Event emission is incorrect, which can cause off-chain indexers/consumers to misattribute values.

3.1.3 Recommendation

```
1 - emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
2 + emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

3.2 [L-2] TSwapPool::swapExactInput() always returns 0

Severity: Low **Status:** Open

3.2.1 Description

`swapExactInput` is expected to return the actual amount of tokens bought by the caller. It declares the named return value `output`, but never assigns it and has no explicit `return` statement.

3.2.2 Impact

The return value will always be 0, giving callers (and any composing contracts) incorrect information about the swap they just executed.

3.2.3 Proof of Concept

```

1 function testOutput() public {
2     vm.startPrank(liquidityProvider);
3     weth.approve(address(pool), 100e18);
4     poolToken.approve(address(pool), 100e18);
5     pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6     vm.stopPrank();
7     vm.startPrank(user);
8     poolToken.approve(address(pool), 1 ether);
9     uint256 output = pool.swapExactInput(
10         poolToken,
11         1 ether,
12         weth,
13         1000,
14         uint64(block.timestamp)
15     );
16     assertEq(output, 0);
17 }
```

3.2.4 Recommendation

```

1 function swapExactOutput(
2     IERC20 inputToken,
3     IERC20 outputToken,
4     uint256 outputAmount,
5     uint64 deadline
6 )
7 public
8 revertIfZero(outputAmount)
9 revertIfDeadlinePassed(deadline)
10 returns (uint256 inputAmount)
11 {
12     uint256 inputReserves = inputToken.balanceOf(address(this));
13     uint256 outputReserves = outputToken.balanceOf(address(this));
14
15     inputAmount = getInputAmountBasedOnOutput(
16         outputAmount,
17         inputReserves,
18         outputReserves
19     );
20     _swap(inputToken, inputAmount, outputToken, outputAmount);
21 + return inputAmount;
22 }
```

4 Informational Findings

4.1 [I-1] PoolFactory::PoolFactory__PoolDoesNotExist is unused and should be removed

```
1 -error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

4.2 [I-2] Lacking zero-address check in PoolFactory constructor

```
1 constructor(address wethToken) {
2 +   if (wethToken == address(0)) revert PoolFactory__WethIsZeroAddress();
3   i_wethToken = wethToken;
4 }
```

4.3 [I-3] PoolFactory::createPool should use .symbol() instead of .name()

```
1 -string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());
2 +string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).symbol());
```

4.4 [I-4] Magic numbers should be replaced with named constants

Affects `getInputAmountBasedOnOutput()`, `getPriceOfOneWethInPoolTokens()`, and `getPriceOfOnePoolTokenInWeth()` in `TSwapPool.sol`.

```
1 -((inputReserves * outputAmount) * 10000) /
2 - ((outputReserves - outputAmount) * 997);
3 +uint256 private constant FEE_PRECISION = 1000;
4 +uint256 private constant FEE_MULTIPLIER = 997;
```

4.5 [I-5] TSwapPool::swapExactInput() should be external

```
1 function swapExactInput(
2     IERC20 inputToken,
3     uint256 inputAmount,
4     IERC20 outputToken,
5     uint256 minOutputAmount,
6     uint64 deadline
7 )
8 - public
9 + external
10 revertIfZero(inputAmount)
11 revertIfDeadlinePassed(deadline)
12 returns (uint256 output)
13 {}
```

4.6 [I-6] TSwapPool::swapExactOutput() is missing NatSpec for the deadline parameter

```
1 * @param inputToken ERC20 token to pull from caller
2 * @param outputToken ERC20 token to send to caller
3 * @param outputAmount The exact amount of tokens to send to caller
4 + * @param deadline The deadline for the transaction to be completed by
5 */
6 function swapExactOutput(
7     IERC20 inputToken,
8     IERC20 outputToken,
9     uint256 outputAmount,
10    uint64 deadline
11 )
```

4.7 [I-7] TSwapPool::_swap() uses deprecated contract-type equality comparison

Comparing variables of contract type is deprecated and scheduled for removal; cast to `address` and compare explicitly.

```
1 -if (
2 -    _isUnknown(inputToken) ||
3 -    _isUnknown(outputToken) ||
4 -    inputToken == outputToken
5 -)
6 +if (
7 +    _isUnknown(inputToken) ||
8 +    _isUnknown(outputToken) ||
9 +    address(inputToken) == address(outputToken)
10 +)
```

5 Gas / Static Analysis Notes (Aderyn)

Automated static analysis via [Aderyn](#) surfaced the following, in addition to the manual findings above:

- `public` functions not used internally (e.g. `swapExactInput`) could be marked `external` — see I-5.
- Repeated literal values (`997`, `1e18`) should be declared as named constants — see I-4.
- `PoolCreated`, `LiquidityAdded`, `LiquidityRemoved`, and `Swap` events are missing `indexed` fields, reducing off-chain query efficiency.
- Solidity 0.8.20 targets the Shanghai EVM version by default, which emits `PUSH0` — confirm target chain(s) support this opcode before deployment.
- Large literals (e.g. `10000`) can be expressed in scientific notation (`1e4`) for readability.
- The unused `PoolFactory__PoolDoesNotExist` custom error should be removed — see I-1.

6 Disclaimer

This report is provided for informational purposes only and does not constitute a guarantee of security. A security audit is not a warranty of bug-free code, and does not guarantee the complete absence of vulnerabilities. The findings reflect the codebase as of the commit hash listed above; any changes made afterward are outside the scope of this report.